

SCOPE INDIA

THIRUVANANTHAPURAM • KERALA • INDIA

INTERNSHIP PROJECT REPORT

*CI/CD Deployment of a Spring Boot REST API on Amazon EKS
with Jenkins, Docker, Docker Hub & Amazon RDS MySQL*

Submitted by Sreerag K

Internship Duration January 2026 – July 2026

Organization Scope India, Thiruvananthapuram

Project Repository github.com/Sreerux/project3-source

Report Date July 2026

This report documents a complete DevOps CI/CD pipeline for deploying a Spring Boot REST API on Amazon EKS using Jenkins. The application is containerized with Docker, stored on Docker Hub, connected to Amazon RDS MySQL, and automatically deployed to Kubernetes whenever code changes are pushed to GitHub. All server addresses, repository names, and pipeline stages referenced in this document reflect the actual working environment used for the project.

1. Project Overview & Objectives

This project demonstrates a complete DevOps CI/CD pipeline for deploying a Spring Boot REST API on Amazon EKS using Jenkins. The application is containerized with Docker, stored on Docker Hub, connected to AWS RDS MySQL, and automatically deployed to Kubernetes whenever code changes are pushed to GitHub.

Objectives

- Automate application build and deployment.
- Deploy a Spring Boot application on Kubernetes.
- Store Docker images in Docker Hub.
- Connect the application to AWS RDS MySQL.
- Reduce manual deployment effort using Jenkins.

Project Workflow

- 1 Developer pushes code to GitHub.
- 2 Jenkins pulls the latest source code.
- 3 Maven builds the Spring Boot application.
- 4 Docker builds the application image.
- 5 Docker image is pushed to Docker Hub.
- 6 Jenkins updates the Kubernetes deployment.
- 7 Amazon EKS deploys new pods.
- 8 Application connects to AWS RDS.
- 9 Users access the application through the Load Balancer.

Verified working: This workflow was run end-to-end on Jenkins job **Project-3**, build #8, and completed with every stage passing, ending in a confirmed rollout to Amazon EKS.

2. Technologies & AWS Services Used

Category	Tool
Cloud	AWS
Source Control	GitHub
CI/CD	Jenkins
Build Tool	Maven
Containerization	Docker

Category	Tool
Container Registry	Docker Hub
Orchestration	Amazon EKS
Database	Amazon RDS (MySQL)
CLI	kubectrl
OS	Ubuntu Linux

AWS Services Used

- **IAM** — identities and permissions for Jenkins to interact with EKS.
- **EC2** — hosts the Jenkins server and the EKS worker nodes.
- **EKS** — managed Kubernetes control plane running the application.
- **RDS** — managed MySQL database storing application data.
- **VPC** — networking layer connecting EC2, EKS, and RDS.
- **Security Groups** — firewall rules controlling access between EKS and RDS.
- **Load Balancer** — AWS ELB automatically provisioned by the Kubernetes Service, exposing the application publicly.

Infrastructure snapshot

Instance	Type	Status	Role
jenkins-sever	c7i-flex.large	Running	Jenkins CI/CD server
mycluster-clus...	c7i-flex.large	Running	EKS worker node
mycluster-clus...	c7i-flex.large	Running	EKS worker node
k8s	c7i-flex.large	Running	Auxiliary Kubernetes/testing instance

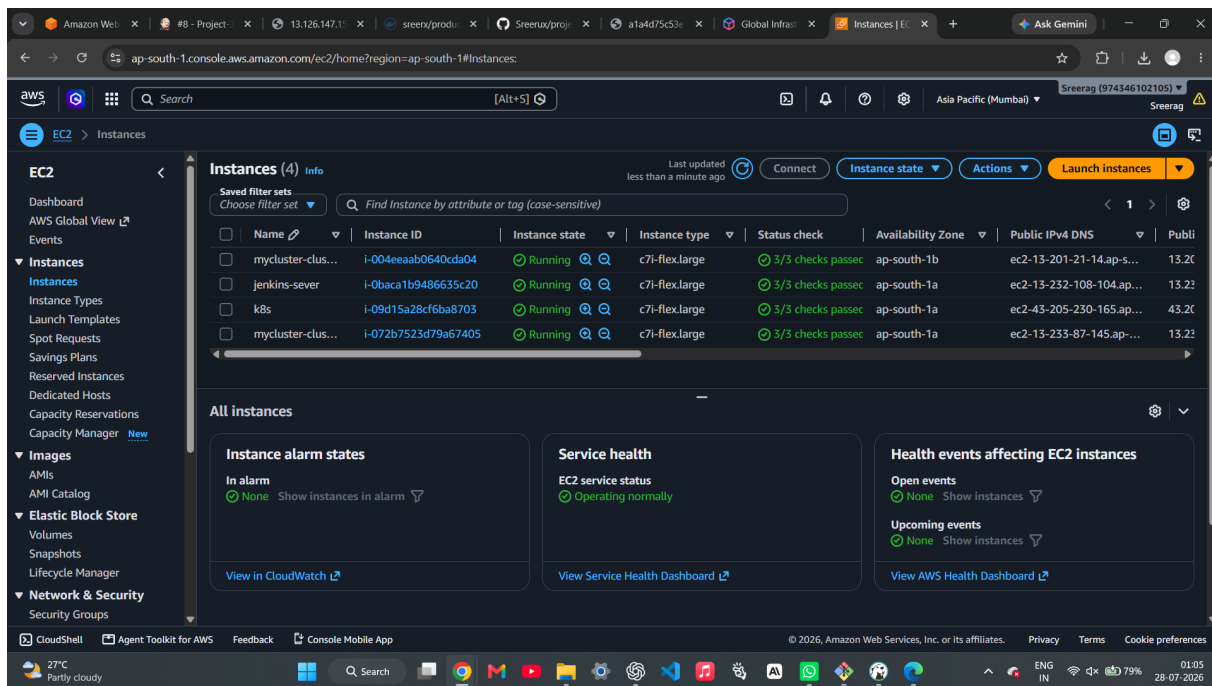
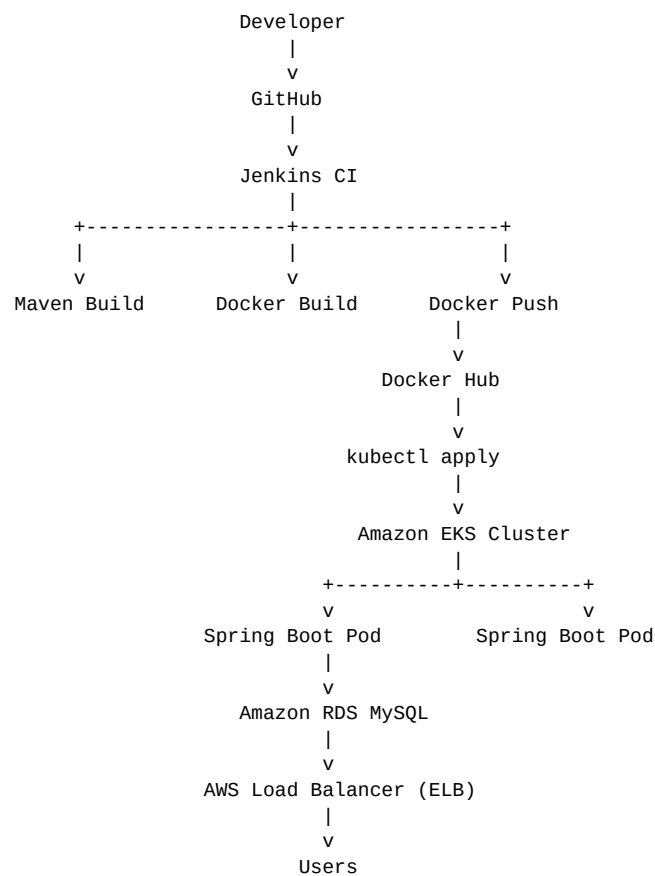
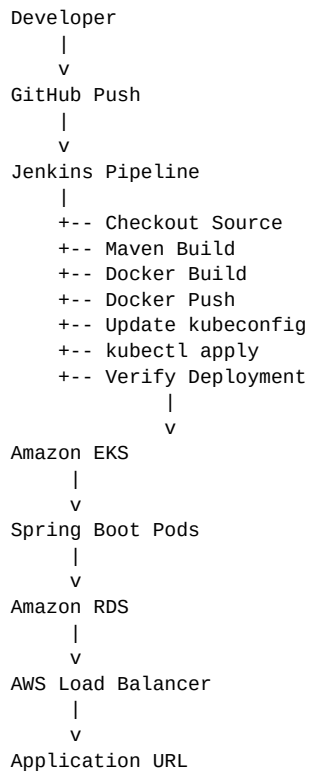


Figure 1: AWS EC2 console (Asia Pacific / Mumbai region) showing all four running instances used across this and the related project — the Jenkins server and two EKS worker nodes underpin this pipeline.

3. Architecture Diagram



CI/CD Pipeline Flow



4. Prerequisites

- A Jenkins server (EC2 instance) with Git, Maven, Docker, and the AWS CLI installed.
- An existing Amazon EKS cluster (**mycluster-cluster4**, region **ap-south-1**) reachable from the Jenkins host.
- An Amazon RDS MySQL instance already provisioned, with its security group allowing inbound connections from the EKS worker nodes.
- A Docker Hub account and credential stored in Jenkins for authenticated image pushes.
- The repository **github.com/Sreerux/project3-source**, containing a Jenkinsfile, Dockerfile, pom.xml, and Deployment.yml at its root.

5. Jenkins Server Configuration

Jenkins runs on the **jenkins-sever** EC2 instance, with JDK21 and Maven3 registered under *Manage Jenkins* → *Tools* — both referenced directly in the Jenkinsfile's `tools{}` block and confirmed by the “Tool Install” stage passing on every run.

Credentials configured in Jenkins

- **dockerhub-creds** — a username/password credential used in the “Build & Push Docker Image” stage, injected as `$DOCKER_USER / $DOCKER_PASS`.

- AWS access credentials available to the Jenkins environment, used by the AWS CLI during `aws eks update-kubeconfig` to authenticate against the EKS cluster.

6. Maven Build Stage

The “Build Application” stage compiles and packages the Spring Boot application, skipping tests for a faster CI cycle, then lists the resulting artifact for confirmation:

```
mvn clean package -DskipTests
ls -lh target/
```

A separate “Debug Dockerfile” stage runs immediately before the build, printing the Dockerfile contents and a full repository listing — a lightweight sanity check that catches a missing or misconfigured Dockerfile before it can break the image build:

```
echo "==== Dockerfile ====="
cat Dockerfile
echo "==== Repository Files ====="
ls -R
```

7. Containerizing the Application

The application is built into a Docker image and pushed to Docker Hub in a single combined stage, authenticating with credentials injected via `withCredentials`:

```
docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest
echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
docker push ${IMAGE_NAME}:${IMAGE_TAG}
docker push ${IMAGE_NAME}:latest
```

Where **IMAGE_NAME** is **sreerx/products-api** and **IMAGE_TAG** is the Jenkins build number (**8** for the run documented here). The application listens on container port **8082** — an earlier port mismatch against 8081 caused an “empty reply from server” error, resolved by aligning the Dockerfile’s exposed port with the Kubernetes Deployment and Service definitions (see Section 12, Troubleshooting Guide).

Verified working: Build #8’s “Build & Push Docker Image” stage completed in 17 seconds with `docker build -t sreerx/products-api:8` . followed by a successful push of both tags.

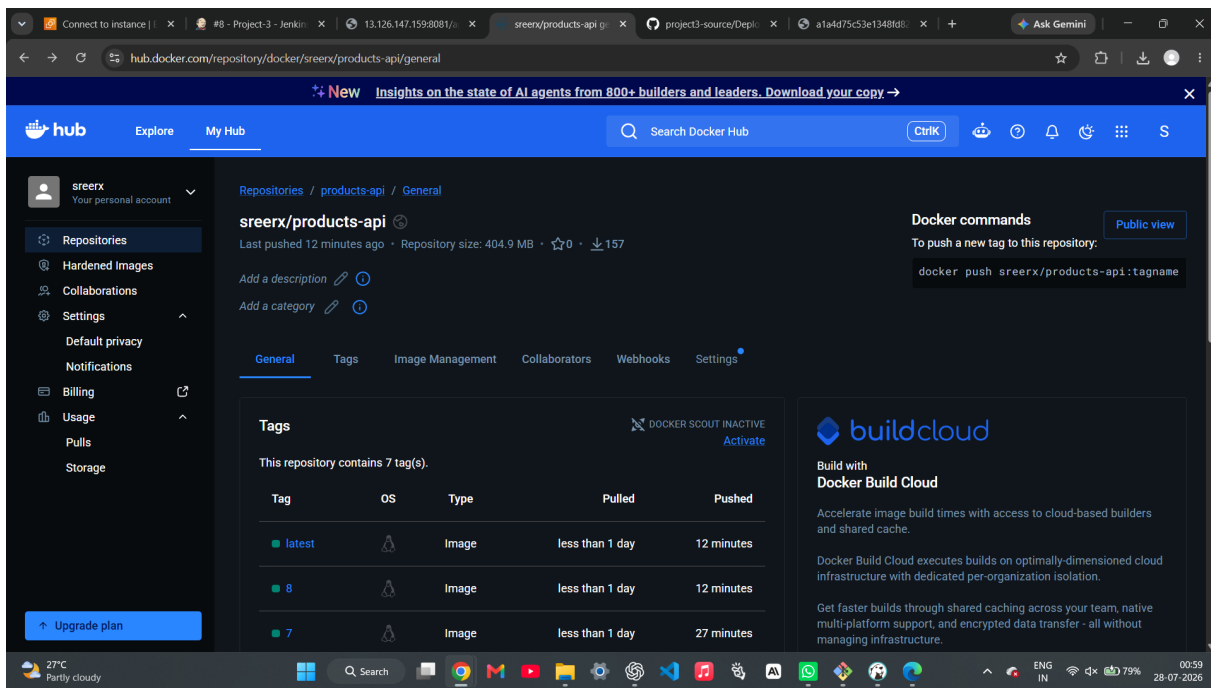


Figure 2: Docker Hub repository sreerx/products-api, showing the latest and :8 tags pushed moments apart by this pipeline run, alongside earlier build tags.

8. Kubernetes Deployment Manifest

The manifest defines two replicas of the application container behind a LoadBalancer-type Service, matching the live cluster state (**products-api** deployment, **products-api-service** Service):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: products-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: products-api
  template:
    metadata:
      labels:
        app: products-api
    spec:
      containers:
        - name: products-api
          image: sreerx/products-api:latest
          ports:
            - containerPort: 8082
---
apiVersion: v1
kind: Service
metadata:
  name: products-api-service
spec:
  type: LoadBalancer
  selector:
    app: products-api
  ports:
    - port: 80
```

targetPort: 8082

*Note: This YAML is reconstructed from the live kubectl get all output (replica count, resource names, and the corrected port 8082) rather than transcribed from the repository's **Deployment.yml** file directly, since its raw contents weren't included with the screenshots shared.*

Kubernetes resources at a glance

- **Deployment** — creates the application pods, maintains the desired replica count (2), and performs rolling updates whenever a new image is deployed.
- **Service (type LoadBalancer)** — exposes the application externally and automatically provisions an AWS Elastic Load Balancer.

9. Amazon RDS Integration

The Spring Boot application connects to Amazon RDS MySQL using JDBC, at **products-db.cjyu66i2w9gl.ap-south-1.rds.amazonaws.com**. Storing data in RDS rather than inside the cluster means application data survives pod restarts, rescheduling, or a full redeployment.

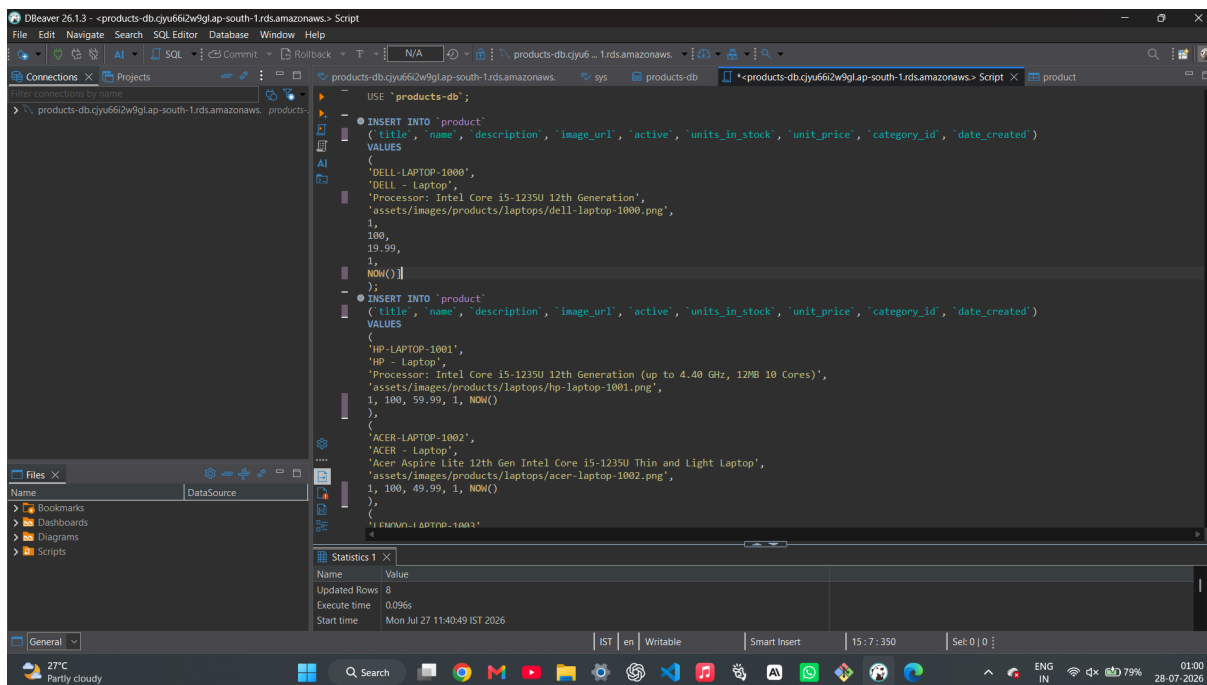


Figure 3: Sample product data being inserted directly into the products-db RDS instance via DBeaver, confirming the database schema (product table with title, name, description, price, stock, and category fields) that the application reads from.

The live API response below confirms the application is reading this exact data back out through its REST endpoint, served through the Load Balancer:

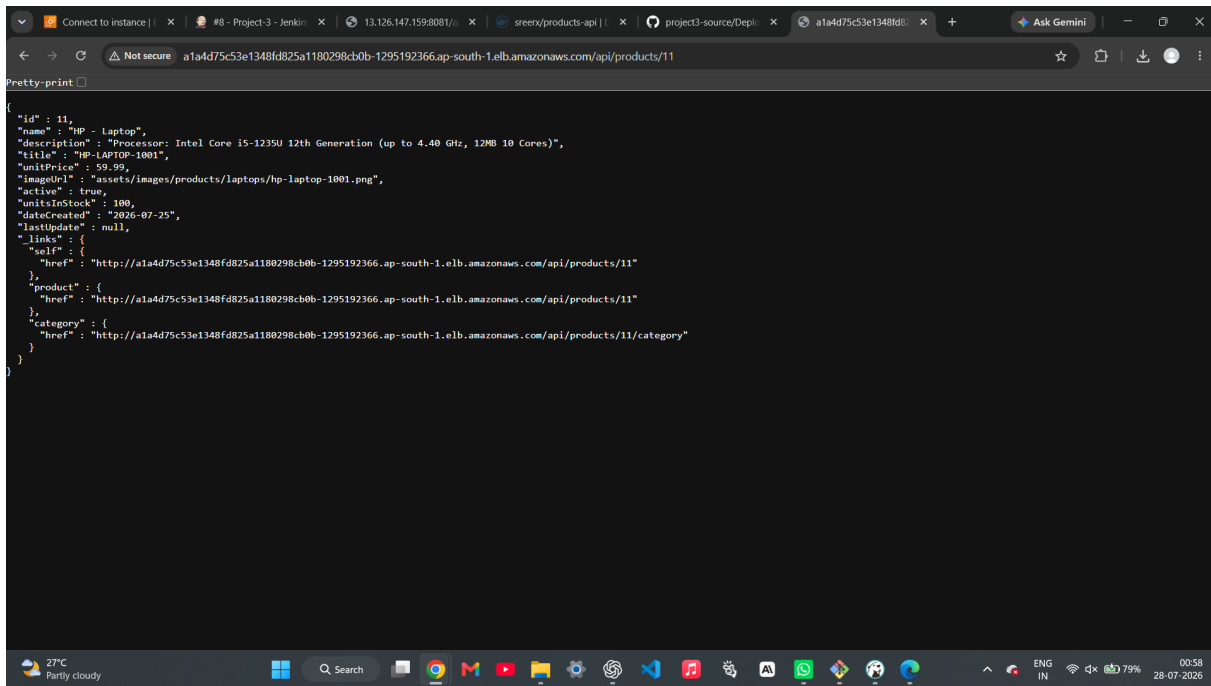


Figure 4: GET /api/products/11 returning the HP-LAPTOP-1001 record with matching field values, confirming the Spring Boot application is correctly reading from Amazon RDS in production.

10. The Complete Jenkins Pipeline

This is the exact Jenkinsfile used for job **Project-3**, confirmed against build **#8**'s successful console output.

```
pipeline {
  agent any
  tools {
    jdk 'JDK21'
    maven 'maven3'
  }
  environment {
    IMAGE_NAME = "sreerx/products-api"
    IMAGE_TAG = "${BUILD_NUMBER}"
    AWS_REGION = "ap-south-1"
    CLUSTER_NAME = "mycluster-cluster4"
  }
  stages {
    stage('Checkout') {
      steps {
        cleanWs()
        git branch: 'main',
            url: 'https://github.com/Sreerux/project3-source.git'
      }
    }
    stage('Verify Source') {
      steps {
        sh '''
        pwd
        ls -la
        '''
      }
    }
    stage('Debug Dockerfile') {
      steps {
        sh '''
        echo "===== Dockerfile ====="
        '''
      }
    }
  }
}
```

```

        cat Dockerfile
        echo "==== Repository Files ====="
        ls -R
        '''
    }
}
stage('Build Application') {
    steps {
        sh '''
            mvn clean package -DskipTests
            ls -lh target/
            '''
    }
}
stage('Build & Push Docker Image') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )
        ]) {
            sh '''
                docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
                docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest
                echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
                docker push ${IMAGE_NAME}:${IMAGE_TAG}
                docker push ${IMAGE_NAME}:latest
                '''
        }
    }
}
stage('Deploy to EKS') {
    steps {
        sh '''
            aws eks update-kubeconfig \
            --region ${AWS_REGION} \
            --name ${CLUSTER_NAME}
            kubectl get nodes
            kubectl apply -f Deployment.yml
            kubectl rollout restart deployment/products-api
            kubectl rollout status deployment/products-api
            '''
    }
}
stage('Verify Deployment') {
    steps {
        sh '''
            kubectl get deployments
            kubectl get pods -o wide
            kubectl get svc
            '''
    }
}
}
post {
    success {
        echo "Application Successfully Deployed to Amazon EKS"
    }
    failure {
        echo "Pipeline Failed"
    }
    always {
        cleanWs()
    }
}
}

```

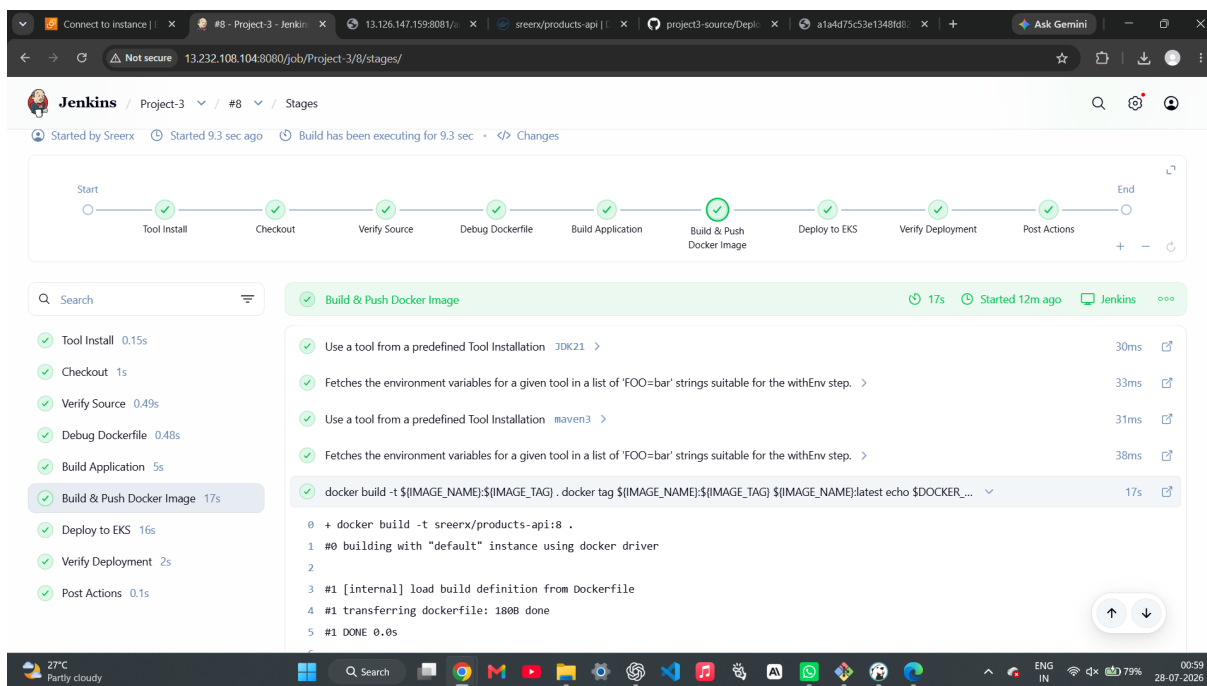


Figure 5: Jenkins Stage View for Project-3, build #8, showing all nine stages — Tool Install, Checkout, Verify Source, Debug Dockerfile, Build Application, Build & Push Docker Image, Deploy to EKS, Verify Deployment, and Post Actions — completed successfully, with the live console log for the Docker stage visible on the right.

11. GitHub Repository Structure

The source repository follows a clean, professional layout that separates application code, deployment manifests, database setup, and documentation:

```
products-api/
|
|■■■■ Jenkinsfile
|■■■■ Dockerfile
|■■■■ pom.xml
|■■■■ README.md
|
|■■■■ kubernetes/
|■■■■ deployment.yaml
|■■■■ service.yaml
|
|■■■■ sql/
|■■■■ DB_Setup.sql
|
|■■■■ screenshots/
|■■■■ jenkins-success.png
|■■■■ dockerhub-image.png
|■■■■ eks-pods.png
|■■■■ eks-service.png
|■■■■ application-output.png
|
|■■■■ src/
```

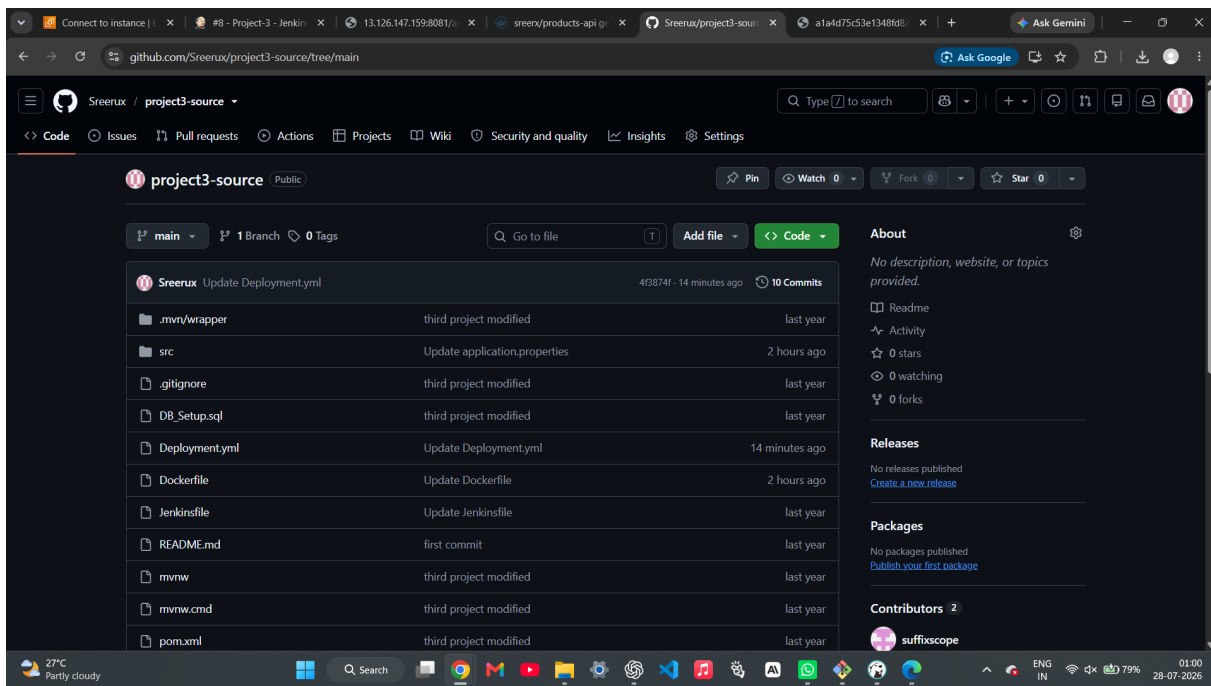


Figure 6: The actual GitHub repository (Sreerux/project3-source) file listing, including Jenkinsfile, Dockerfile, pom.xml, DB_Setup.sql, and Deployment.yml at the repository root.

12. Verifying the Live Deployment

After the pipeline finished, the deployment was confirmed directly from the Jenkins/EKS-connected host using kubectl, both for the running pods and the externally reachable Service:

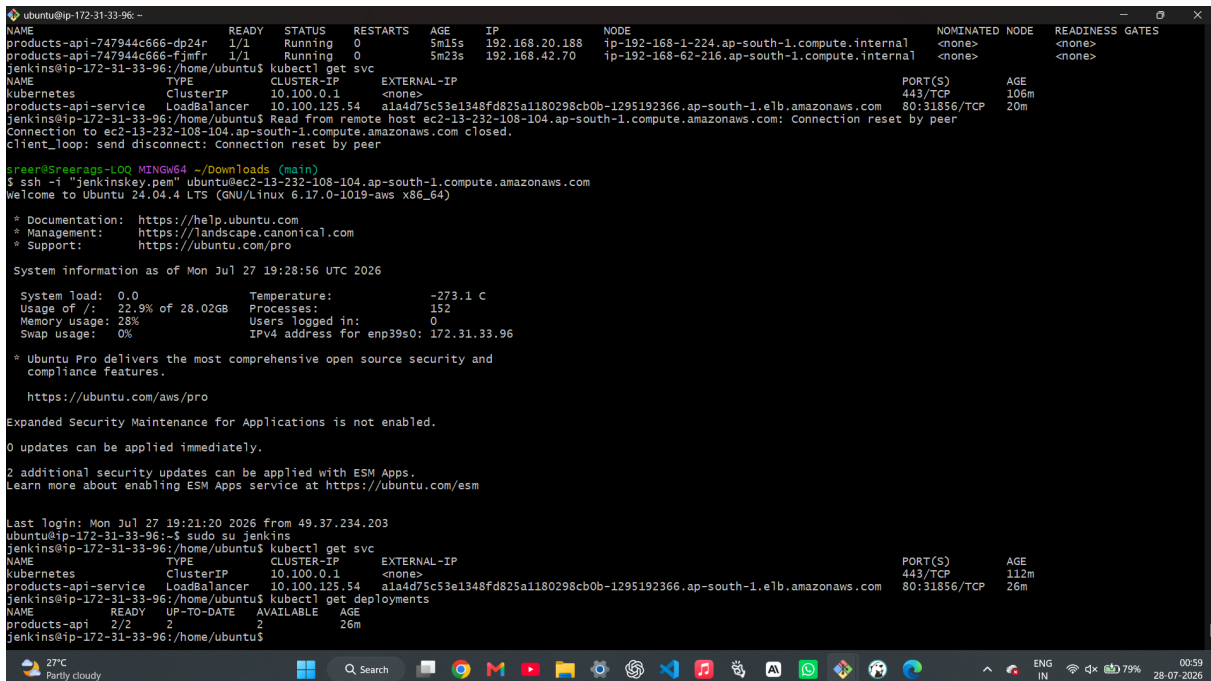


Figure 7: Terminal output confirming 2 of 2 products-api pods Running across two worker nodes, and the products-api-service LoadBalancer Service holding a public AWS ELB address, alongside the corresponding kubectl get deployments output.

The application was reached externally through the AWS Load Balancer endpoint automatically provisioned for the Service:

`http://a1a4d75c53e1348fd825a1180298cb0b-1295192366.ap-south-1.elb.amazonaws.com/api/products/11`

Note: The JSON response confirming this endpoint is live and correctly reading from RDS is shown earlier as Figure 4, in Section 9.

13. Final Pipeline Run — Stage Summary

Stage	Result	Detail
Tool Install	Success	JDK21 and maven3 resolved
Checkout	Success	Cloned Sreerux/project3-source, branch main
Verify Source	Success	Repository contents confirmed present
Debug Dockerfile	Success	Dockerfile and full file tree printed for review
Build Application	Success	mvn clean package -DskipTests completed
Build & Push Docker Image	Success	sreerx/products-api:8 and :latest built and pushed (17s)
Deploy to EKS	Success	kubeconfig updated, Deployment.yml applied, rollout restarted (16s)
Verify Deployment	Success	2/2 pods Running, Service confirmed (2s)
Post Actions	Success	Workspace cleaned

Build #8 finished in roughly 41 seconds end-to-end, started by user **Sreerx**.

14. Troubleshooting Guide

Issue	Cause	Resolution
Docker permission denied	Jenkins user not in Docker group	Added Jenkins to Docker group
Maven build failed	Missing dependencies	Verified pom.xml and rebuilt
Image push failed	Incorrect Docker credentials	Updated Docker Hub credentials in Jenkins
EKS authentication failed	kubeconfig not updated	Ran aws eks update-kubeconfig
Pods not starting	Wrong image or configuration	Checked pod logs and corrected deployment
Application inaccessible	Incorrect container port	Updated Deployment and Service port configuration
LoadBalancer pending	AWS resource creation delay	Waited for ELB provisioning
Empty reply from server	Port mismatch (8081 vs 8082)	Updated Kubernetes Deployment and Service to port 8082
Database connection issues	Security Group or JDBC URL	Allowed EKS access to RDS and verified configuration
Deployment not updating	Old image cached	Used kubectl rollout restart deployment/products-api

15. Project Summary

This project successfully implements a complete DevOps CI/CD pipeline using GitHub, Jenkins, Maven, Docker, Docker Hub, Amazon EKS, Kubernetes, and Amazon RDS. Code changes are automatically built, containerized, pushed to Docker Hub, and deployed to Amazon EKS through Jenkins. The application is exposed using an AWS Load Balancer and connected to an external MySQL database hosted on Amazon RDS. This demonstrates modern cloud-native deployment practices, infrastructure automation, and continuous delivery.

Project Outcome

- Fully automated CI/CD pipeline.
- Dockerized application.
- Kubernetes deployment on Amazon EKS.
- External access using an AWS Load Balancer.
- Database hosted on Amazon RDS.

16. Reference Links (Quick Copy)

- Project repository: github.com/Sreerux/project3-source
- Jenkins downloads: jenkins.io/download
- Jenkins Pipeline syntax: jenkins.io/doc/book/pipeline/syntax
- Maven downloads: maven.apache.org/download.cgi
- Docker Engine install: docs.docker.com/engine/install
- Docker Hub: hub.docker.com
- Amazon EKS: aws.amazon.com/eks
- Amazon RDS for MySQL: aws.amazon.com/rds/mysql
- kubectl install: kubernetes.io/docs/tasks/tools/#kubectl
- AWS CLI: aws.amazon.com/cli